

Algorithm Design Paradigms

Handout on Non-Recursive DP

07 April 2026

Last time our DP was focused on taking a recursive solution and remembering previous calls to speed it up. This time, we will take this up a notch and talk about non recursive DP.

Note, this is not all the DP ideas. Far from it. These are the ideas in the intersection of things I was aware of, was comfortable giving and has some real chance of appearing on the quiz.

§1 Warm Up

☰ Blackbird's Ship

Ace has been captured by Blackbeard and is held prisoner aboard the Saber of Xebec, a massive ship with n cabins. Blackbeard is demanding a hefty ransom.

Nami, negotiating on behalf of the Straw Hats, agreed to the following terms. She will guess the cabin number where Ace is held. If she guesses correctly, Blackbeard releases Ace immediately. Otherwise, if her guess is too high, Blackbeard informs her of this and she pays him a Berries. If her guess is too low, she pays b Berries.

We want to compute the minimum cost Nami can incur using the optimal strategy, irrespective of where Ace is imprisoned. Can you find an $O(n)$ algorithm to compute this?

Hint: Consider $dp[l]$ that represents the maximum sized ship we can search at cost of l berries.

§2 The Problems

☰ Travelling Rockstar

Given n cities and a table T such that $T[i][j]$ is time it takes to go from city i to city j .

A rockstar wants to have a tour covering all the n cities. Can you find a tour that minimizes the travel time?

Note: We want a $O(n^2 2^n)$ algorithm.

☰ All Pairs Shortest Path

Given a weighted graph $G = (V, E)$, design an algorithm to compute the shortest path distances between every pair of vertices in V .

Note: We would like a $O(|V|^3)$ algorithm.

Hint: The algorithm is dumber than you could think of. It is like so, so dumb!

☰ Single Source Shortest Path

Given a weighted graph with positive weights $G = (V, E)$, a start location S and target location T ; design an algorithm to find the shortest path from S to T .

Note: This algorithm's story is like one of the coolest thing I know of.

☰ Party

You are organizing a party for your office. Unfortunately, the office is very hierarchial and people don't feel comfortable attending a party if their immediate senior is attending (although they have no issues with others. Basically, you and boss of your boss can be invited but not you and your boss).

Given a list S of length n with $S[i]$ being the immediate underlings of i ($S[j]$ being empty indicates j is at the bottom of the hierarchy). Everyone has only one immediate boss.

What is the most number of people you can invite?

Hint: We want a $O(n)$ algorithm

☰ Party with Money

Same setup as **Party** but we need money to even do this party. Every person has an amount of money they are willing to contribute. We want to invite the people, in an attempt to maximize the amount of money we have to party. Can you do that in $O(n)$?

☰ Root Replace DP

Given a tree with n nodes, find a node such that when this node is the root, the sum of the depths of all nodes is maximized.

Note: This can be done in $O(n)$ time as well.

☰ Digital DP

Given positive integers a and b , How many times does each digit appear in all integers $[a, b]$?

Hint: We can solve this in $O(\log_{10}(b))$.

Hint: $9000 \rightarrow 9900 \rightarrow 9990 \rightarrow 9999$ also seems like a valid way to count... doesn't it?

☰ Just a normal DP

Given a positive integer n , determine how many ways it can be expressed as the sum of k positive integers, where different orders are considered distinct partitions.

☰ Another normal DP

Starting with an empty list $[]$, we have two operations.

- Increase all the elements with 1
- Append 1 to the list

Given an n , what is the minimum number of operations we need to make the sum of the list n ?

☰ Oh, that's why they were there!

Given a positive integer n , how many ways are there to partition n into a sum of any number of positive integers? (Different orders are considered the same partition.)

Hint: A strategy similar to **Just a normal DP** will give an $O(n^2)$ solution. Similarly, **Another normal DP** might also provide a $O(n^2)$ solution.

Extension: Looking at the solutions, can you see some part of the solution that uses more time in both? Notice that these are both disjoint. So maybe we can combine both the solutions? This would give an $O(n\sqrt{n})$ solution.

§2.1 Knapsack and Friends

☰ 0-1 Knapsack

Given a bag with capacity C and N items such that the weight of i -th item is $W[i]$ and value is $V[i]$.

Choose a subset of I of $[1, N]$ that maximizes:

$$\sum_{i \in I} V[i]$$

upto the constraint

$$\sum_{i \in I} W[i] \leq C$$

Note: We have seen a solution that took $O(NC)$ space last time. Can you find a solution taking only $O(C)$ space?

☰ Unbounded Knapsack

The model is same as **0-1 Knapsack** with the change that we can choose each item arbitrary number of times. To state formally,

Choose a **multiset** of I of $[1, N]$ that maximizes:

$$\sum_{i \in I} V[i]$$

upto the constraint

$$\sum_{i \in I} W[i] \leq C$$

Note: We wish to find a $O(NC)$ time and $O(C)$ space.

☰ Bounded Knapsack

The model is same as **0-1 Knapsack** but now for the i -th item, there is a number $B[i]$ that is the stock of item i or how many time you can add it to the bag.

Formally, choose a **multiset** of I of $[1, N]$ that maximizes:

$$\sum_{i \in I} V[i]$$

upto the constraints

$$\sum_{i \in I} W[i] \leq C$$

$$\forall i \in N, |\{x = i \mid x \in I\}| \leq B[i]$$

Note: We want a $O(CN \log(\max(B)))$ algorithm. There is an obvious $O(C \sum(B)) = O(CN \max(B))$ algorithm which you could start with.

☰ Mixed Knapsack

Let's say we have all the above types of items. Can you find a way to solve this efficiently?

☰ Grouped Knapsack

Let's go back to **0-1 Knapsack**.

This time there are groups of items and we can take only one from each group. Can you find an efficient algorithm that solves this?